QuantSolve

A self-verifying, self-healing solver for quantitative problems

Generative AI Laboratory, Lab CA

Self-Correction 'Code Execution & Auto-Heal' Loop. Concept: ReAct (Reason + Act) / Reflexion Loops.
Task: build a Python execution loop that generates code for a complex task, runs it in a safe execution environment, captures tracebacks on failure and iteratively fixes the code.

Name	Roll No	Batch
Deon Menezes	16014223030	B1
Aishwarya Gawali	16014223007	B1

GitHub repository: https://github.com/DeonMenezes-11/QuantSolve

Date: 06 October 2026

Execution engine: restricted `exec()` inside a child process | Model: gpt-4.1-mini (OpenAI API)

1. Aim

To design, build and evaluate a Python execution loop that takes a quantitative word problem, asks a large language model to write a program for it, runs the program in a restricted `exec()` sandbox, captures the traceback or failed self-check when something goes wrong, and repairs the program over several attempts using the ReAct and Reflexion ideas.

2. Objectives

- Generate complete Python programs from natural-language problems under a strict contract: the program computes everything, stores its answer in a `RESULT` dictionary and contains at least two `assert` self-checks.
- Execute untrusted generated code with layered defences: an AST safety gate, restricted builtins with a guarded import, a separate child process with a scrubbed environment, a CPU limit, a memory watchdog and a wall-clock timeout.
- Classify every failure (blocked, syntax, runtime, assertion, timeout, contract) and return a cleaned traceback to the model as the Observation.
- Add Reflexion-style verbal self-reflection stored in an episodic memory, plus a loop guard that notices repeated code or repeated errors.
- Provide a command-line interface and a web application (FastAPI backend and a hand-built single-page front end) that stream each Thought, Action, Observation and Reflection live, and save a full JSON trace of every run.
- Measure the effect of the loop against one-shot generation on eight problems with hand-written reference answers, including runs where a bug is deliberately injected, and test the sandbox against known escape and resource-exhaustion attacks.

3. Theory

3.1 LLM code generation and why a feedback loop is needed

Large language models write code that usually looks right. For numerical problems the typical failure is not a wild hallucination but a small slip: a misspelled variable, a loop that runs one step too few, a sign error, a value that is rounded too early, or a library call with the wrong arguments. In one-shot generation nothing notices these slips; the program either crashes or, worse, prints a plausible but wrong number. Executing the code gives the system an external, objective signal (the interpreter does not care how confident the model sounded), and feeding that signal back lets the model correct itself.

3.2 Program-aided reasoning (PAL, Program of Thoughts)

PAL (Gao et al., 2023) and Program of Thoughts (Chen et al., 2022) separate reasoning from calculation: the model writes the reasoning as a program and the Python interpreter performs the arithmetic. This removes the arithmetic errors that language models make when they compute in text, and it makes the reasoning inspectable. QuantSolve follows this pattern: the model never states the final number itself; it only writes the program that computes it.

3.3 ReAct: reasoning and acting

ReAct (Yao et al., 2023) interleaves free-text reasoning traces with actions in an environment, and uses the observation returned by each action to decide the next step. In QuantSolve one attempt is one ReAct step: the *Thought* is the model's short plan (method, formulas, which invariants it will check), the *Action*

is the program, and the *Observation* is the sandbox result (a RESULT dictionary, or a failure kind with a cleaned traceback).

3.4 Reflexion and Self-Refine

Reflexion (Shinn et al., 2023) improves an agent across trials without changing any weights. After a failed trial the model writes a short verbal reflection on what went wrong; the reflection is kept in an episodic memory and added to the prompt of later trials. Self-Refine (Madaan et al., 2023) is a related loop in which the same model gives feedback on its own output and then refines it. QuantSolve uses a separate reflection call after every failure that returns a diagnosis, a lesson and a next plan; all reflections so far are shown to every later attempt.

3.5 Self-verification with assertions

The interpreter only reports crashes. A program that runs to completion with a logic error produces a wrong answer with no traceback at all. To turn silent errors into observable ones, the code contract requires at least two assertions that compare quantities computed in different ways, for example a month-by-month loan simulation against the closed-form total interest, or a dynamic-programming count against a brute-force count. A failed assertion becomes an Observation like any other error. The weakness is that the same model writes the checks: a check can itself be wrong (a false alarm) or too weak to notice a real error (a silent pass). Both effects show up in the results.

3.6 Sandboxing with exec()

Python's `exec()` with a custom globals dictionary is convenient but it is not a security boundary: through object introspection (for example `().__class__.__base__.__subclasses__()`), generator frames or modules that re-export `os` and `sys`, code can reach the rest of the interpreter. The practical answer is defence in depth: reject dangerous constructs before running (static AST analysis), give the code only a curated set of builtins, run it in a separate process that holds no secrets, and bound its CPU time, memory and wall time so that it cannot hang or exhaust the host.

4. System architecture

Figure 1 shows the loop. A task goes to the solver prompt, which returns a thought and a program in JSON mode. In demo or benchmark runs the fault injector can corrupt the first program. The program then passes through the AST safety gate, runs in a fresh child process and is checked against the contract. If it passes, its RESULT is summarised in plain English. If any stage fails, the failure becomes the Observation, a reflection is written and stored in memory, and the next attempt receives the previous code, the observation, the whole memory and, if the loop guard fired, a nudge to change approach.

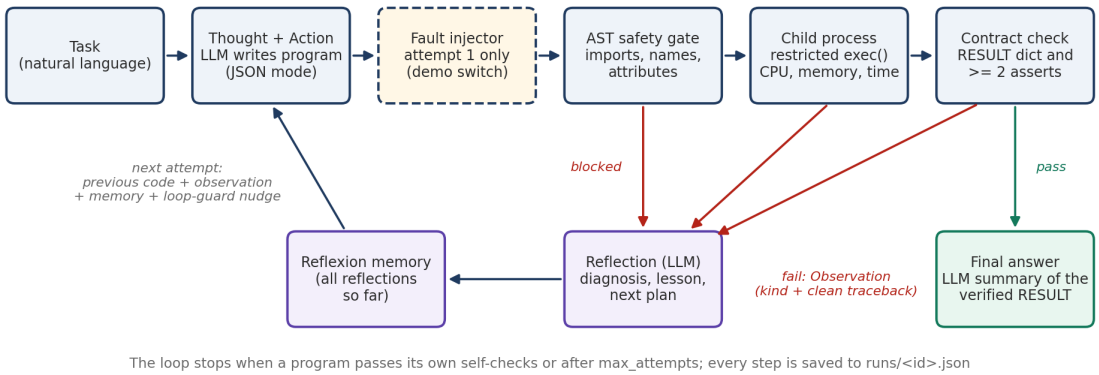


Figure 1. The QuantSolve loop: ReAct steps (Thought, Action, Observation) with Reflexion memory between attempts.

5. Implementation

The project is written in Python 3.13 with module-level functions and dataclasses rather than class hierarchies, so that the core package can be read in one sitting. Table 1 lists the modules.

Module	Lines	Responsibility
`config.py`	44	Settings dataclass loaded from .env (model, max attempts, timeout, temperatures).
`llm.py`	84	OpenAI chat wrapper in JSON mode with a simple retry and token/cost accounting (gpt-4.1-mini: $0.40 / $1.60; gpt-4o-mini: $0.15 / $0.60 per million input / output tokens).
`prompts.py`	124	Solve, reflect and summarise prompts, and the formatting of observations and memory.
`sandbox.py`	437	AST safety gate, restricted builtins, child-process runner, failure classification and traceback cleaning.
`faults.py`	159	Deterministic AST-based bug injection (crash and logic modes).
`agent.py`	269	The ReAct + Reflexion loop, loop guard, events for the UI, and JSON traces.
`tasks.py`	337	Eight problems with hand-written reference solutions and scoring.
`benchmark.py`, `safety_eval.py`	297	The two evaluation scripts.
`cli.py`, `web.py`	218	Rich command-line interface; launcher for the website.
`webapp/*.py`	478	FastAPI server: JSON API, Server-Sent Events stream of loop events, background solve jobs with a stop flag, read-only data loaders.
`webapp/static/`	2778	Hand-written HTML, CSS and JavaScript front end (no framework, no build step): live run view, history, results, explainer.

Table 1. Modules of the `quantsolve` package and the two front ends (line counts measured when this report was built).

5.1 Sandbox design

Execution happens in three stages. **(1) Static checks in the parent.** The code is parsed; a syntax error is reported with its line. The AST gate then rejects imports outside the allow-list (bisect, cmath, collections, datetime, decimal, fractions, functools, heapq, itertools, math, numpy, random, statistics), relative and star imports, the names `eval, exec, compile, open, __import__, input, breakpoint, exit, quit, globals, locals, vars, getattr, setattr, delattr, memoryview`, any attribute starting with an underscore (this covers every dunder as well as private module attributes such as `random._os`), a list of attributes that lead out of the sandbox

without a dunder (frame and generator internals such as `gi_frame` and `f_back`, modules re-exported by allowed modules such as `statistics.sys`, and numpy file I/O such as `load` and `tofile`), and format strings that read such attributes. A test walks every public attribute reachable from the allowed modules and confirms that no dangerous module can be reached. Programs with fewer than two assert statements are rejected as a contract violation before they run.

(2) A fresh child process. The parent starts a `multiprocessing` child with the "spawn" start method; the target function lives in `sandbox.py` so it can be pickled from the CLI, from the web server's worker threads, or from standard input. The child first deletes every environment variable except a few harmless ones (so the OpenAI key is not present), sets `RLIMIT_CPU`, starts a watchdog thread that kills the process if its resident memory exceeds 1 GB (macOS ignores memory rlimits), redirects stdout and stderr, and calls `exec(compile(code, "<solution>", "exec"), ns)` where `ns` holds only `SAFE_BUILTINS`. That dictionary contains common functions and exception classes, `__build_class__` so that classes work, and a guarded `__import__` that re-checks the allow-list at run time (it has to allow private sub-modules of allowed packages, because C code such as `ndarray.mean` imports them lazily through the calling frame's builtins). **(3) Classification.** The child sends back a small dictionary over a pipe: success flag, RESULT (checked to be a JSON-serialisable dict), truncated stdout, exception type and message, and a traceback restricted to frames in `<solution>` with each source line pulled from the code string. The parent waits for a "ready" message and then applies the timeout to the user code only; if it expires the child is terminated and then killed.

5.2 Failure taxonomy

Kind	Raised when	Example observation
blocked	The AST gate finds a forbidden import, name, attribute or string.	line 1: import of 'os' is not allowed
syntax	The code does not parse.	SyntaxError: '(' was never closed (line 1)
runtime	Any exception other than AssertionError; also the memory watchdog.	NameError: name 'annual_raet' is not defined. Did you mean: 'annual_rate'?
assertion	A self-check failed: the program is internally inconsistent.	AssertionError: interest mismatch: 5413573.61 vs 5413878.80
timeout	Wall-clock limit (default 10 s) or the CPU rlimit was hit.	Execution exceeded the 10 s time limit and the process was killed
contract	Fewer than 2 asserts, RESULT missing, not a dict, or not JSON.	RESULT is not JSON-serialisable (Object of type set is not JSON serializable)

Table 2. Failure kinds returned to the model as the Observation.

5.3 Prompts

All prompts live in `prompts.py` and use JSON mode. The **solve** prompt states the code contract: allowed imports only; no file, network or introspection features; compute every number in code; assign the requested keys to `RESULT` with plain JSON values; and write at least two assertions that compare quantities obtained in different ways, with tolerances for floats and the values in the message. The model returns `{"thought", "code"}`. On later attempts the prompt also carries the previous program, the observation and the numbered Reflexion memory, and asks for a complete corrected program that fixes the root cause without weakening the checks. The **reflect** prompt asks for `{"diagnosis", "reflection", "next_plan"}` in at most two sentences each, citing the failing line, and reminds the model that a failed assertion means two quantities disagree, so it must decide which side is wrong rather than delete the check. The **summarise** prompt turns the verified

RESULT into a short plain-English answer without recomputing anything.

5.4 Loop guard and fault injection

After each failure the agent hashes the normalised AST of the code (so comment or whitespace changes do not count as new code) and forms an error signature (kind, exception type, line). If either has been seen before, the next prompt carries a nudge to change approach and the solver temperature is raised for that call; for repeated assertion failures the nudge suggests checking both sides of the assertion on a tiny case. The fault injector, used only on attempt 1, rewrites the program with `ast.NodeTransformer`: *crash* renames one use of a variable to a misspelling (NameError), and *logic* makes the first `for ... in range()` loop stop one step early, or flips the first + or - inside a loop. Logic faults rarely crash (an index error or an endless loop is possible), so usually only the program's own assertions can notice them.

5.5 Key code: the loop

The excerpt below is read from `quantsolve/agent.py` when this report is built (event emission and bookkeeping lines included).

```
for number in range(1, settings.max_attempts + 1):
    if stop_requested():
        trace.stopped = True
        break
    emit("attempt_start", {"attempt": number})
    thought, code = solve_step(task, settings, usage, last_code, last_obs, memory, nudge)
    llm_code, injected = None, None
    if number == 1 and inject:
        llm_code = code
        code, injected = faults.mutate(code, inject)
    emit("action", {"attempt": number, "thought": thought, "code": code,
                    "llm_code": llm_code, "injected": injected})
    if code.strip():
        obs = sandbox.run(code, timeout_s=settings.timeout_s)
    else:
        obs = ExecutionResult(False, "contract", "The response did not contain any code.")
    attempt = Attempt(number, thought, code, obs.to_dict(), llm_code, injected)
    trace.attempts.append(attempt)
    emit("observation", {"attempt": number, "observation": attempt.observation,
                         "usage": usage.to_dict()})
    if obs.ok:
        break
    # Loop guard: identical code or an identical error signature -> nudge next prompt.
    fingerprint, signature = code_fingerprint(code), error_signature(obs, code)
    nudge = None
    if fingerprint in seen_code:
        nudge = NUDGE_SAME_CODE
    elif signature in seen_errors:
        nudge = NUDGE_SAME_ERROR.format(sig=f"{obs.kind} {obs.exc_type or ''} line {obs.line}")
        if obs.kind == "assertion":
            nudge += NUDGE_ASSERT_HINT
    seen_code.add(fingerprint)
    seen_errors.add(signature)
    attempt.nudge = nudge
    if stop_requested():
        trace.stopped = True
        break
    if number < settings.max_attempts:  # no point reflecting after the last attempt
        reflection = reflect_step(task, code, attempt.observation, memory, settings, usage)
        reflection["attempt"] = number
        attempt.reflection = reflection
        memory.append(reflection)
        emit("reflection", {"attempt": number, "reflection": reflection, "nudge": nudge,
                            "usage": usage.to_dict()})
    last_code, last_obs = code, attempt.observation
```

6. Walkthrough of real runs

6.1 An injected bug that was caught and healed

This section follows run `20261004-235610_sip_stepup_inject-logic` (model gpt-4.1-mini), taken from the saved JSON trace. The task was *Step-up SIP future value*; the full problem text is in `quantsolve/tasks.py`. A **logic** fault was injected into attempt 1, and the run finished after 2 attempts using 4 LLM calls, 5,526 tokens and about $0.0041.

Because the fault is injected after the model has written the program, it is fair to ask whether attempt 1 failed because of the fault or because of a bug the model wrote itself. Re-running the model's original attempt-1 program offline gives the answer: it passes every self-check, so the failure below is caused by the injected fault.

Attempt 1, Thought. I will compute the monthly instalments by applying the 10% annual step-up each year, then calculate the future value by compounding each instalment from its month of investment to the end at 1% monthly. The total invested is the sum of all instalments. The wealth gained is future value minus total invested. For fv_without_stepup, I will assume a constant Rs 10,000 monthly instalment for 180 months and compound similarly. I will assert that total invested equals the sum of instalments computed two ways (yearly step-up sum vs monthly sum), and that the future value computed by summing individual instalments' compounded values matches the formula for fv_without_stepup when step-up is zero.

Injected fault. logic: off-by-one in range() on line 7: `range(1, months + 1)` -> `range(1, months)`

The executed program around the corrupted line was:

```
  4  base_instalment = 10000
  5  step_up_rate = 1.1
  6  instalments = []
  7  for m in range(1, months):
  8      year = (m - 1) // 12
  9      instalment = base_instalment * step_up_rate ** year
 10      instalments.append(instalment)
 11  total_invested = sum(instalments)
 12  future_value = 0.0
 13  for m, inst in enumerate(instalments, start=1):
```

Observation 1 (kind *assertion*):

```
Traceback (solution frames only):
  line 22, in <module>
    assert abs(total_invested - total_invested_check) < 1e-06, f'Total invested mismatch: {total_investe
    d} vs {total_invested_check}'
AssertionError: Total invested mismatch: 3774722.81994046 vs 3812697.8032987844
```

Reflection 1 (stored in memory). *Diagnosis:* The loop on line 7 uses range(1, months), which iterates from 1 to 179, missing the 180th instalment, causing total_invested (line 11) to be underestimated compared to total_invested_check (line 21). *Lesson:* You must carefully check loop bounds to ensure all instalments are included, especially when the problem specifies a fixed number of periods; off-by-one errors in loops can cause subtle but significant mismatches in sums. *Next plan:* Change the loop on line 7 to range(1, months + 1) to include the 180th instalment and recalculate total_invested accordingly.

Attempt 2, Thought. The original code missed the last instalment by using range(1, months) instead of range(1, months + 1), causing the total invested to be underestimated. I fixed this by including the 180th instalment in the loop. This ensures total_invested matches the sum computed by yearly aggregation. I kept the other logic intact and the self-checks verify total invested and future values computed by two methods.

The change between the failing program and the final program (unified diff, comments and blank lines ignored):

```
--- attempt 1 (executed)
+++ attempt 2
@@ -6,3 +6,3 @@
 instalments = []
-for m in range(1, months):
+for m in range(1, months + 1):
     year = (m - 1) // 12
```

Observation 2: all 2 self-checks passed and the program returned `{"total_invested": 3812698, "future_value": 8683849, "wealth_gained": 4871152, "fv_without_stepup": 5045760}`.

Scored against the hand-written reference, the result is **correct** (total_invested: expected 3812698, got 3812698; future_value: expected 8683849, got 8683849; wealth_gained: expected 4871152, got 4871152; fv_without_stepup: expected 5045760, got 5045760).

Final answer. The total amount invested over 15 years with the 10% annual step-up is Rs 3,812,698. The future value of this investment at the end of 15 years is Rs 8,683,849. The wealth gained, which is the difference between the future value and the total invested, amounts to Rs 4,871,152. If the investor had invested Rs 10,000 every month without any step-up, the future value would have been Rs 5,045,760.

6.2 An injected bug that slipped past the self-checks

Run `20261004-235320_compound_varying_inject-logic` (Compound interest with changing rates and deposits) shows the most important failure mode. The injector made this change: logic: off-by-one in range() on line 20: `range(2020, 2025)` -> `range(2020, 2024)`. The mutated program still passed all 3 of its self-checks on attempt 1, so the loop had nothing to react to: it stopped and returned a wrong answer. Table 3 compares the returned RESULT with the reference and with the model's original (un-mutated) program, re-run offline.

RESULT key	Reference	Returned (with the bug)	Model's original program
`final_value`	391962.57	317295.38 (wrong)	391962.57
`total_interest`	91962.57	17295.38 (wrong)	91962.57
`equivalent_rate_pct`	6.784	1.405 (wrong)	6.784

Table 3. A silent failure: the injected bug changed the answer but not the outcome of the checks.

The self-checks in that program were:

```
assert abs(total_interest - (final_value - total_deposits)) < 1e-06, f'Interest mismatch: {total_interes
    t} vs {final_value - total_deposits}'
assert abs(gf_2020_formula - gf_2020_calc) < 1e-12, f'2020 growth factor mismatch: {gf_2020_formula} vs
    {gf_2020_calc}'
assert abs(final_from_equiv - final_value) < 0.01, f'Final value mismatch: {final_from_equiv} vs {final_
    value}'
```

The model's original program gets 3 of 3 keys right, so its method was sound and the injected bug alone made the answer wrong. The checks it wrote were true of both the correct and the corrupted program, which is exactly the weakness of self-written assertions: they only protect the quantities they actually compare.

6.3 A run that did not heal

Run `20261004-235548_loan_prepay_inject-logic` (Home loan with yearly prepayments, logic fault) shows the limits of the loop. It used all 5 attempts without passing. The injected fault was: logic: flipped +/- inside a loop on line 15: `emi - interest` -> `emi + interest`. Table 4 lists what each attempt observed.

#	Observation	Reflection: diagnosis (shortened)	Loop guard
1	*assertion*: AssertionError: Principal repaid no prepay 5002209.376464678 != 5000000	The principal repaid in amortization_no_prepayment (line 15) is incorrectly calculated as emi + interest instead of emi - interest, causing principal_no_prepay to exceed ...	-
2	*assertion*: AssertionError: Principal repaid with prepay 3700000.0000000005 != 5000000	In amortization_with_prepayment (lines 22-45), the prepayment is subtracted after incrementing months and updating balance, but the loop condition and final payment logic...	-
3	*assertion*: AssertionError: Principal repaid with prepay 3700000.0000000005 != 5000000	The amortization_with_prepayment function (line 27) does not re-check the loop condition after applying the prepayment at line 44, so the loop may exit prematurely before...	You submitted code that is identical to an earlier attempt
4	*assertion*: AssertionError: Principal repaid with prepay 3700000.0000000005 != 5000000	In amortization_with_prepayment (lines 27-48), the months counter is incremented after applying the prepayment (line 45), but the prepayment is applied only if (months + ...	The same error (assertion AssertionError line 58) happened again
5	*assertion*: AssertionError: Principal repaid with prepay 3700000.0000000005 != 5000000	-	The same error (assertion AssertionError line 63) happened again

Table 4. Observations of the run that did not heal.

The first reflection pointed at line 15, where the fault had been injected. Attempt 2 no longer failed on the first check, but a different check then failed and kept failing. Re-running the model's original attempt-1 program offline shows that it also failed, so the check that kept failing was the model's own problem, present before the fault was injected. The loop guard fired, but the model kept repeating the same approach.

7. Results

7.1 Setup

The benchmark uses the 8 problems in Table 5. Each is a multi-step problem whose text states every rule and the exact RESULT keys and units, and each has a hand-written reference solution that is cross-checked by an independent method in the test suite (for example a year-by-year closed form for the loan, a generating function for the birthday triples, an RK4 simulation for the projectile). A run is *correct* only if every scored key is within its tolerance. Every case was run 2 time(s) with model gpt-4.1-mini, at most 5 attempts and a 10 s sandbox timeout. The arms are: **one-shot** (one attempt, no healing), **loop** (the full QuantSolve loop), **loop + logic bug** (a logic fault injected into attempt 1) and **loop + crash bug** (a crash fault on attempt 1, run on 3 tasks).

Id	Problem	Scored RESULT keys
`loan_prepay`	Home loan with yearly prepayments	emi, months_with_prepayment, months_saved, interest_saved
`sip_stepup`	Step-up SIP future value	total_invested, future_value, wealth_gained, fv_without_stepup
`income_tax`	Indian income tax, new regime FY 2024-25	tax_a, tax_b, tax_c, effective_rate_c_pct
`projectile_drag`	Best launch angle with linear air drag	best_angle_deg, max_range_m, range_at_45_m
`birthday`	Birthday problem: pairs and triples (exact)	n_pair, p_pair, n_triple, p_triple

Id	Problem	Scored RESULT keys
`eoq_discount`	Order quantity with quantity discounts	eoq_base, best_order_qty, min_total_cost, orders_per_year
`compound_varying`	Compound interest with changing rates and deposits	final_value, total_interest, equivalent_rate_pct
`constrained_strings`	Counting strings with adjacency constraints	count_mixed, count_three_digits, prob_three_digits

Table 5. Benchmark problems.

7.2 Overall comparison

Arm	Runs	Correct	Verified	Verified but wrong	Avg attempts	Healed	Tokens	Cost (USD)
one-shot	16	11 (69%)	12	1	1.00	0	37,254	0.0389
loop	16	13 (81%)	16	3	1.19	2	51,313	0.0459
loop + logic bug	16	8 (50%)	16	8	1.88	7	98,482	0.0786
loop + crash bug	6	5 (83%)	5	0	3.00	5	67,838	0.0485

Table 6. Benchmark summary (gpt-4.1-mini). Verified: the final program passed its own self-checks. Verified but wrong: it passed them but disagreed with the reference. Healed: attempt 1 failed and a later attempt passed.

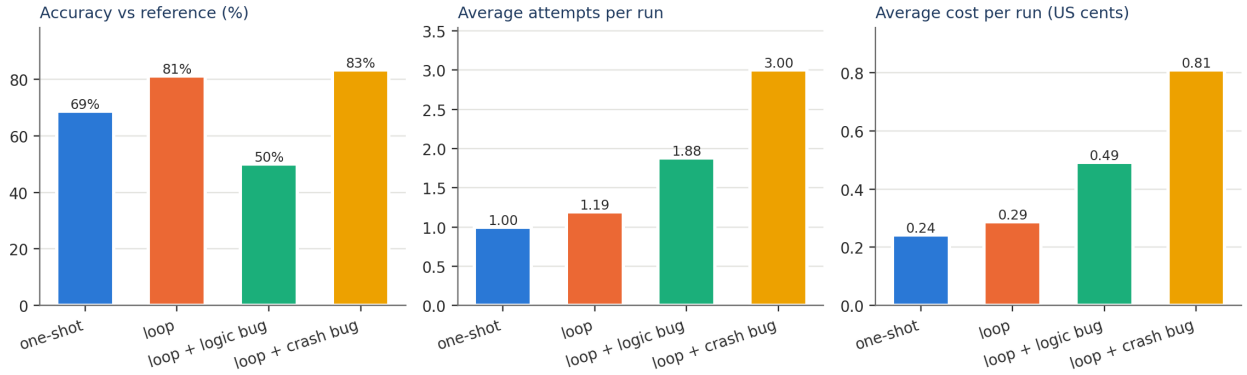


Figure 2. Accuracy, average attempts and average cost per run for each arm.

Across 16 runs per arm, the loop raised accuracy by 12 percentage points, from 69% for one-shot generation to 81%. It needed 1.19 attempts per run on average and healed 2 run(s) whose first program failed. The price is cost: a loop run cost 0.29 US cents on average against 0.24 cents for one-shot. In the one-shot arm 4 of 16 first programs failed in the sandbox (most often *assertion*), which is the room the loop has to work with. On the other hand 3 loop run(s) ended with a program that passed its own checks but was still wrong: the assertions only test what the model thought to test.

With a logic bug injected into attempt 1, the first program failed in 7 of 16 runs (the injector found a place to mutate in 14). Not every such failure was caused by the fault, because the model's own program often had problems too; Section 7.4 separates the two. This arm reached 50% accuracy with 1.88 attempts on average. In the crash arm 5 of 6 runs recovered from a failed first attempt and the arm reached 83% accuracy; for a NameError the cleaned traceback names the line and even suggests the correct variable, so the repair itself is easy. The whole benchmark (54 runs) cost $0.2120.

7.3 Model comparison

The same benchmark (same tasks, arms, prompts and 2 repeats) was first run with gpt-4o-mini; those results are kept in `results/archive/`. Table 7 puts the two models side by side.

Model	One-shot	Loop	Loop + logic bug	Loop + crash bug	Loop: verified but wrong	Loop: cost / run (US cents)
gpt-4o-mini	1/16 (6%)	5/16 (31%)	4/16 (25%)	2/6 (33%)	7/16	0.19
gpt-4.1-mini	11/16 (69%)	13/16 (81%)	8/16 (50%)	5/6 (83%)	3/16	0.29

Table 7. Correct runs per arm for both models.

With gpt-4o-mini the loop moved accuracy from 6% to 31% (+25 points); with gpt-4.1-mini it moved from 69% to 81% (+12 points). The raw gain in points is not a fair comparison, because the weaker model starts with far more room to improve. Measured as the share of one-shot errors that the loop turned into correct answers, the loop removed 27% of the errors with gpt-4o-mini and 40% with gpt-4.1-mini, and 7 versus 3 of the 16 loop runs ended verified but wrong. The loop does not add knowledge: it turns a failed check into another try, and the try only helps if the model can read the observation, find the real cause, write a correct fix and write checks that would notice a wrong one. The stronger model did this more often, so the benefit of self-correction depends on the capability of the model inside it. Both models are small and each cell rests on few runs, so the exact percentages are noisy.

7.4 Did the self-checks catch the injected bugs?

A failed first attempt in an injected run does not prove that the checks caught the fault: the model may have written its own bug as well. For every injected run the model's original attempt-1 program was therefore re-run offline. *Caught*: the original passes but the mutated program fails. *Silent*: both pass but the numbers differ, so wrong numbers got through the checks. *Benign*: both pass with identical numbers (the mutated loop did not matter). *Original failed*: the model's own program already failed, so nothing can be attributed to the fault. *Not applied*: the injector found no place to mutate (for example, the code did not parse).

Arm	Caught	Silent	Benign	Original failed	Not applied
loop + logic bug	2	5	2	5	2
loop + crash bug	4	0	0	2	0

Table 8. Attribution of attempt-1 outcomes in the injected-bug arms.

Of the 9 logic faults that could be attributed, the checks caught 2, 5 slipped through silently and 2 changed nothing. This is the most direct measurement of self-verification in the project: here most of the logic bugs that changed the answer slipped through and became verified-but-wrong answers. Crash faults, by contrast, are caught whenever the fault is the first thing to go wrong, because the interpreter itself raises the error.

7.5 Failure kinds and per-task behaviour

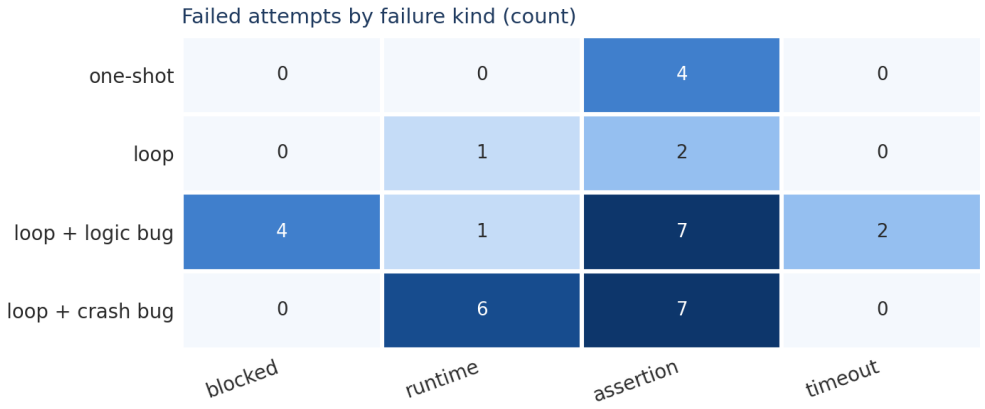


Figure 3. Failed attempts by failure kind for each arm.

Over all arms, 34 attempts failed; the most common kind was *assertion* (20 attempts). assertion: 20, runtime: 8, blocked: 4, timeout: 2. Assertion failures dominating is the intended effect of the contract: problems that would otherwise have produced a quietly wrong number are surfaced as an Observation the model can act on. The other side is that an assertion failure can be a false alarm, when the main computation is right and the model's independent check (often a closed form) is the wrong side.

4 of the *blocked* attempts were not unsafe code: the model used ___ or ____ as a throwaway loop variable and the gate treated it as a dunder name. The benchmark exposed this false positive; it was fixed after the run (a name now counts as a dunder only if it has characters other than underscores) and a regression test was added. The numbers above are from the run before the fix.

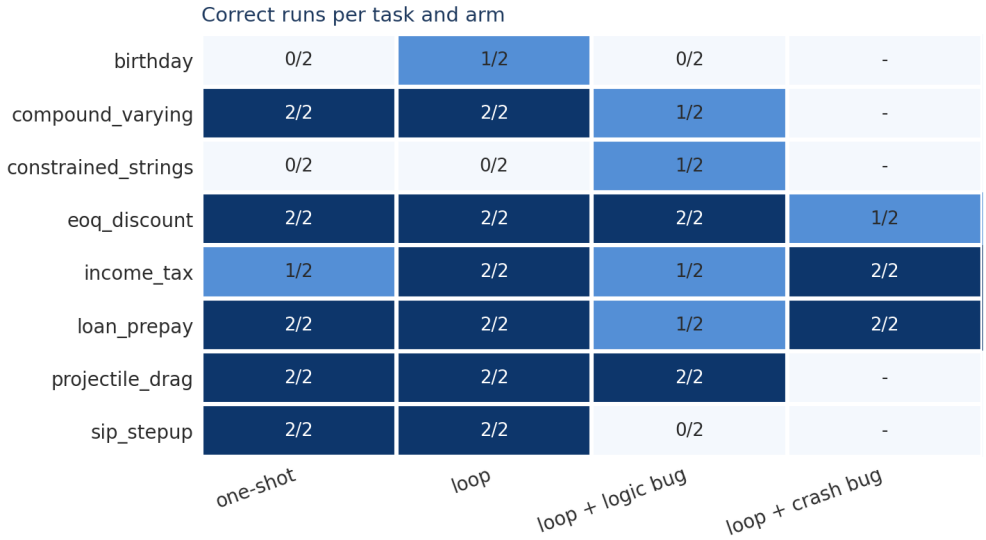


Figure 4. Correct runs out of all runs for each task and arm.

Across the loop arms the tasks solved most reliably were `projectile_drag` (100%), `eoq_discount` (83%), `income_tax` (83%), while the hardest were `birthday` (25%) and `constrained_strings` (25%). Of the 8 loop-arm runs on these two tasks, 6 ended verified but wrong (the checks agreed with a wrong computation, so there was no failing observation for the loop to react to) and 0 never passed their own checks within the attempt budget.

8. Safety evaluation

`safety_eval.py` runs 17 attack programs through the sandbox. Each attack is wrapped in two trivial asserts and a RESULT, so the only reason for a failure is the attack itself. An attack counts as

contained if the expected defence stopped it and no marker file appeared on disk. 3 attacks run with the AST gate switched off to show what the runtime layers do on their own. **17 of 17 attacks were contained.**

Attack	Gate	Outcome	Contained
Shell command through os.system	on	*blocked*: line 1: import of 'os' is not allowed	yes
Read /etc/passwd with open()	on	*blocked*: line 1: name 'open' is not allowed	yes
Classic ().__class__.__base__.__subclasses__() escape	on	*blocked*: line 1: attribute '.__subclasses__' is not allowed	yes
getattr(__builtins__, 'open')	on	*blocked*: line 1: name 'getattr' is not allowed	yes
exec() of a string payload	on	*blocked*: line 1: name 'exec' is not allowed	yes
__import__('subprocess')	on	*blocked*: line 1: name '__import__' is not allowed	yes
random._os (module imported privately by random)	on	*blocked*: line 2: attribute '._os' is not allowed	yes
statistics.sys.modules['os'] (re-exported module)	on	*blocked*: line 2: attribute '.sys' is not allowed	yes
Generator frame walk to the caller's globals	on	*blocked*: line 2: attribute '.f_back' is not allowed	yes
Format-string attribute read '{0.__class__}'	on	*blocked*: line 1: string constants may not contain dunder names	yes
File read through numpy.loadtxt	on	*blocked*: line 2: attribute '.loadtxt' is not allowed	yes
Infinite loop	on	*timeout*: Execution exceeded the 3 s time limit and the process was killed (infinite loop or a very slow algorithm?).	yes
Huge integer power (runs inside C, ignores Python signals)	on	*timeout*: Execution exceeded the 3 s time limit and the process was killed (infinite loop or a very slow algorithm?).	yes
Memory bomb (50 MB chunks forever)	on	*runtime*: MemoryError: the program used more than 1024 MB and was killed.	yes
AST gate disabled: import os (guarded __import__)	off	*runtime*: ImportError: import of 'os' is not allowed in the sandbox	yes
AST gate disabled: open() (restricted builtins)	off	*runtime*: NameError: name 'open' is not defined	yes
AST gate disabled: frame walk reaches os.environ	off	*ok*: {"api_key_visible": false, "env_var_names": ["LC_CTYPE", "PATH", "TMPDIR"]}	yes

Table 9. Safety evaluation (timeout 3 s for this run).

The last row is the most instructive. With the gate off, a generator frame walk does escape the restricted builtins and reaches the real `os.environ` of the child process; the attack is only contained because the child scrubbed its environment before running any user code, so the API key is not there. This is exactly why the restricted `exec()` is treated as one layer among several and not as a security boundary.

9. Limitations and future work

- **The sandbox is not a hard security boundary.** Restricted `exec()` in CPython can be escaped by introspection, which the AST gate blocks by pattern; a new escape route would get through the gate and would then meet only the process boundary and the scrubbed environment. For untrusted users a container, a seccomp/sandbox-exec profile or a WebAssembly Python build would be the right base.
- **Self-checks are written by the same model.** They can be wrong (false alarms that waste attempts and can stop a correct answer from being accepted) or too weak (verified but wrong results). Checks

generated in a separate call, property-based tests, or a second model reviewing the checks would make verification more independent.

- **Small benchmark.** Eight problems with 2 repetition(s) per case give noisy percentages; differences of one or two runs should not be over-read. A larger, more varied set would be needed for firm conclusions.
- **Two small models.** The main runs use gpt-4.1-mini and the comparison uses the archived gpt-4o-mini run, both chosen to keep cost low. Larger models would probably make fewer first-attempt errors and write stronger checks; that was not tested.
- **Memory limit on macOS** is enforced by a watchdog thread that samples peak RSS every 50 ms, so a very fast allocation can overshoot the limit before it is killed.
- Future work: generate the checks in a separate call from the problem statement only (so they cannot copy the program's mistakes), give the model a scratch step to try a formula on a tiny case before writing the final program, and repeat the benchmark with more problems, more repetitions and a stronger model.

10. Conclusion

QuantSolve implements the lab task end to end: an LLM writes a program for a quantitative problem, the program runs in a restricted `exec()` inside a separate process, failures are classified and returned as cleaned observations, and Reflexion-style reflections carry lessons from one attempt to the next. Requiring assert-based self-checks turned some silent logic errors into visible failures, which is what made healing possible for bugs that do not crash; Section 7.4 shows that they also let many through. With gpt-4.1-mini the loop reached 81% accuracy against 69% for one-shot generation, at roughly 1.2 times the cost. With gpt-4o-mini the same loop went from 6% to 31%, so how much self-correction helps depends on how well the base model can diagnose and fix its own mistakes. The sandbox contained 17 of 17 attacks in the safety evaluation. The main lesson is that verification is only as good as the checks: the loop can repair crashes and failed checks, but it cannot repair an error that no check looks at, and it can be trapped by a check that is itself wrong.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.

[2] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan and S. Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023.

[3] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan and G. Neubig. PAL: Program-aided Language Models. ICML 2023.

[4] W. Chen, X. Ma, X. Wang and W. W. Cohen. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. arXiv:2211.12588, 2022 (TMLR 2023).

[5] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh and P. Clark. Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS 2023.

[6] Python Software Foundation. The Python Standard Library: ast, multiprocessing, resource and builtins modules. https://docs.python.org/3/library/

Appendix A. User interface

The web application is started with `python3 web.py` and served by FastAPI on 127.0.0.1. The front end is a single page written in plain HTML, CSS and JavaScript (light and dark themes, usable on a phone). When a problem is submitted the server runs the loop in a background thread and streams every event (attempt start, action, observation, reflection, final answer) to the browser over Server-Sent Events, so each step appears as it happens; a Stop button sets a flag that the loop checks between steps. Saved traces can be replayed at `#/run/<id>`. The screenshots below were taken with headless Chrome driven over the DevTools protocol (`report/capture_ui.py`).

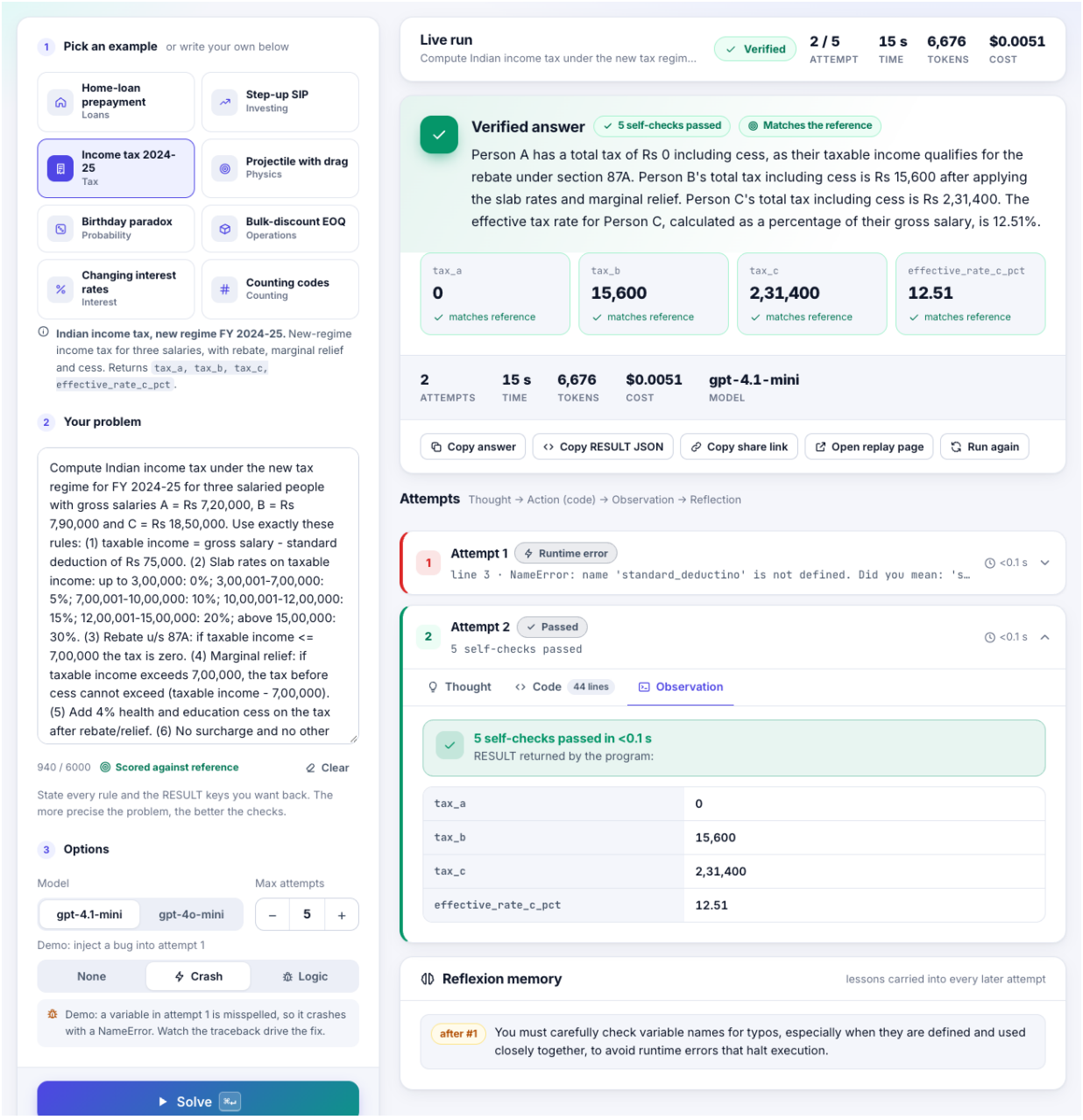


Figure 5. The Solve page after a live run. Left: the example gallery, the problem, the model, the attempt budget and the demo-bug switch. Right: the run header, the verified answer with every RESULT value checked against the hand-written reference, and the attempt timeline (attempt 1 crashed on the injected bug, attempt 2 passed).

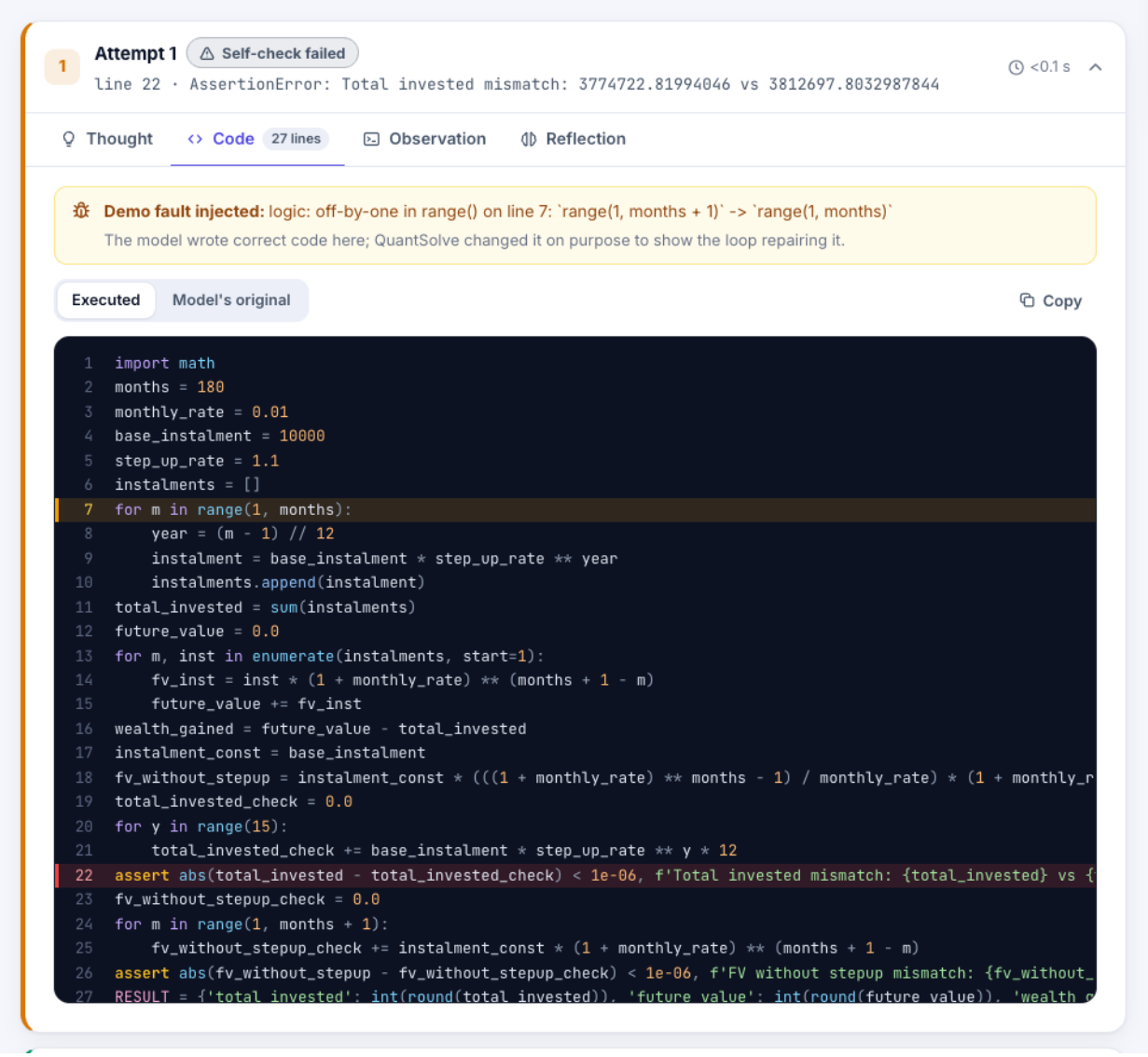


Figure 6. One attempt card (Code tab) from the replay of the walkthrough run: the injected fault is explained in a banner and the failing line is highlighted; the other tabs hold the Thought, the Observation (traceback) and the Reflection.

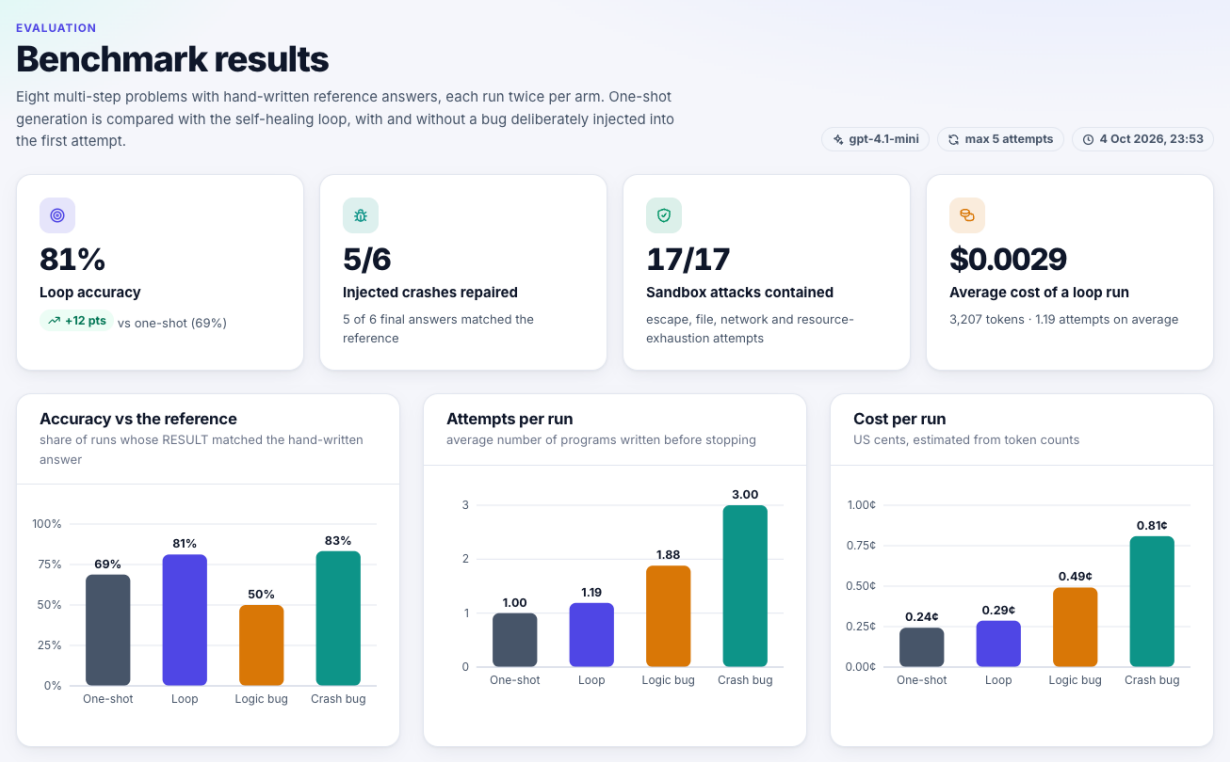


Figure 7. The Results page: headline numbers and per-arm charts read from results/benchmark.json and results/safety.json; further down are the model comparison, a clickable task-by-arm heatmap, the injected-bug attribution and the safety table.

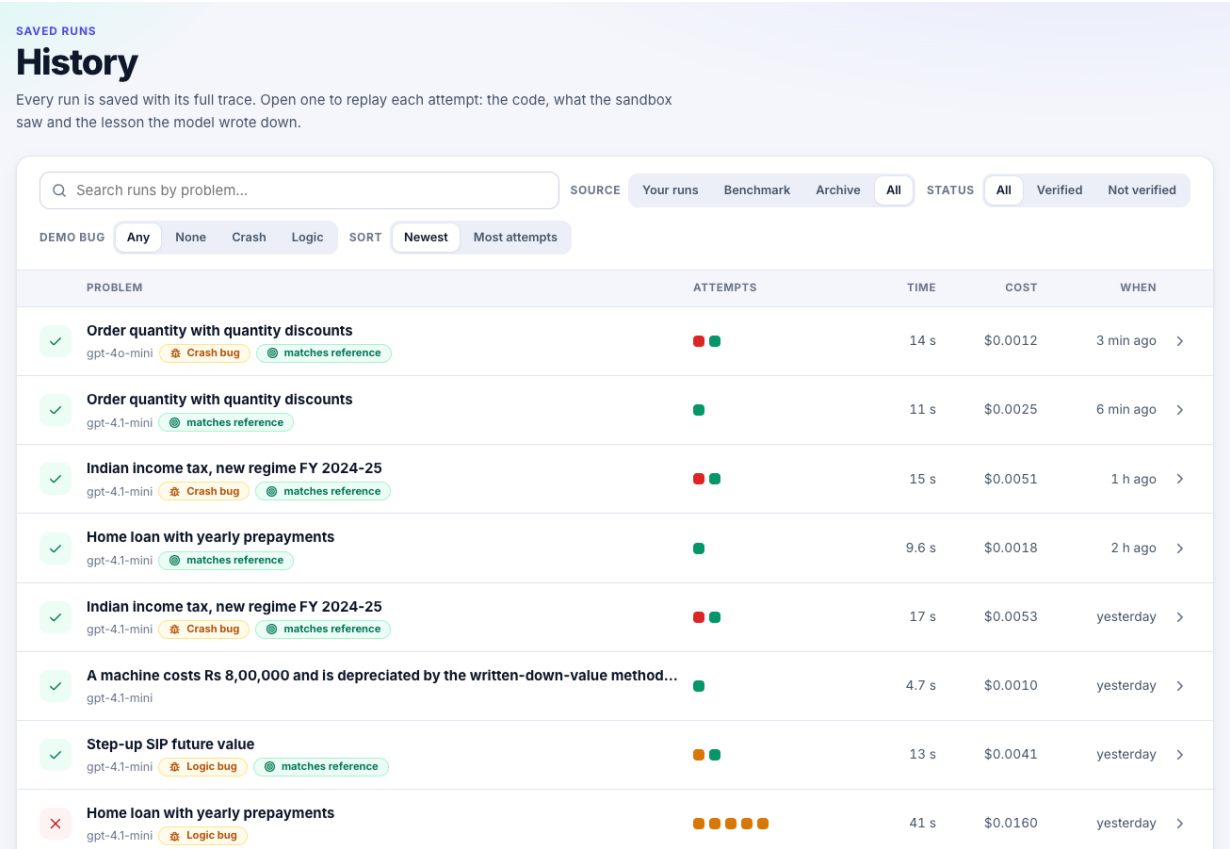


Figure 8. History: every saved run with search and filters. The coloured squares show each run's path through its attempts (red: crash, amber: failed self-check, green: passed).

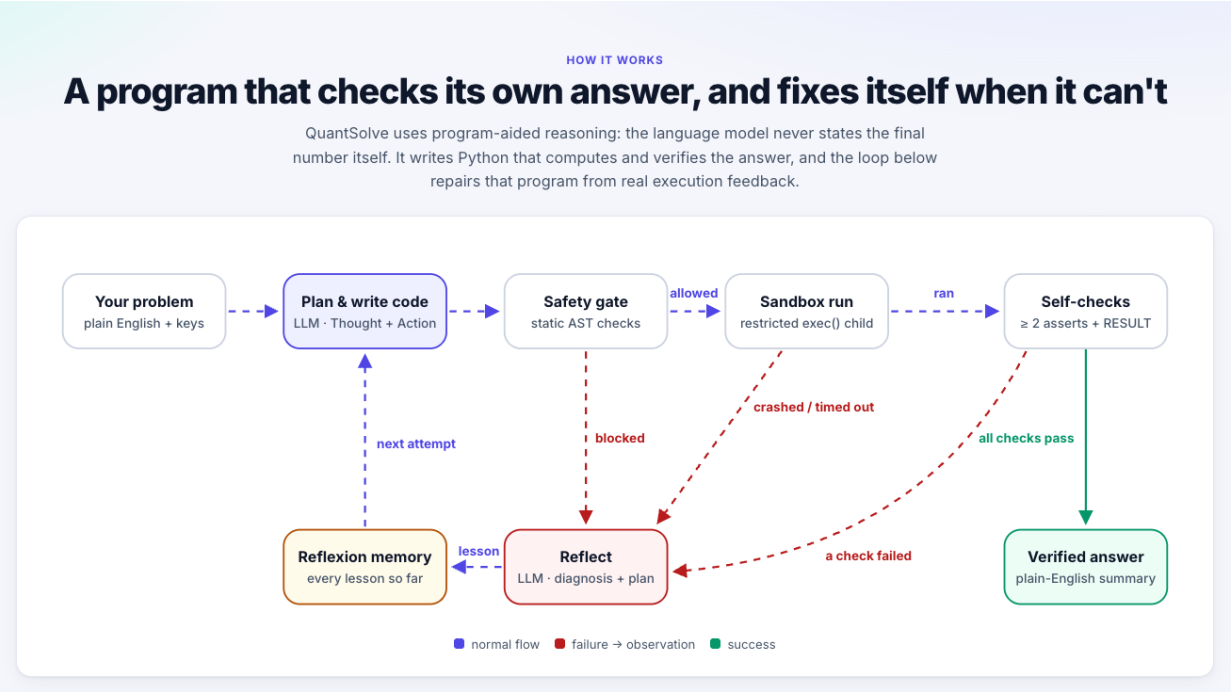


Figure 9. How it works: the loop drawn as a diagram, followed on the page by the three ideas behind it, the six sandbox layers and the failure kinds.

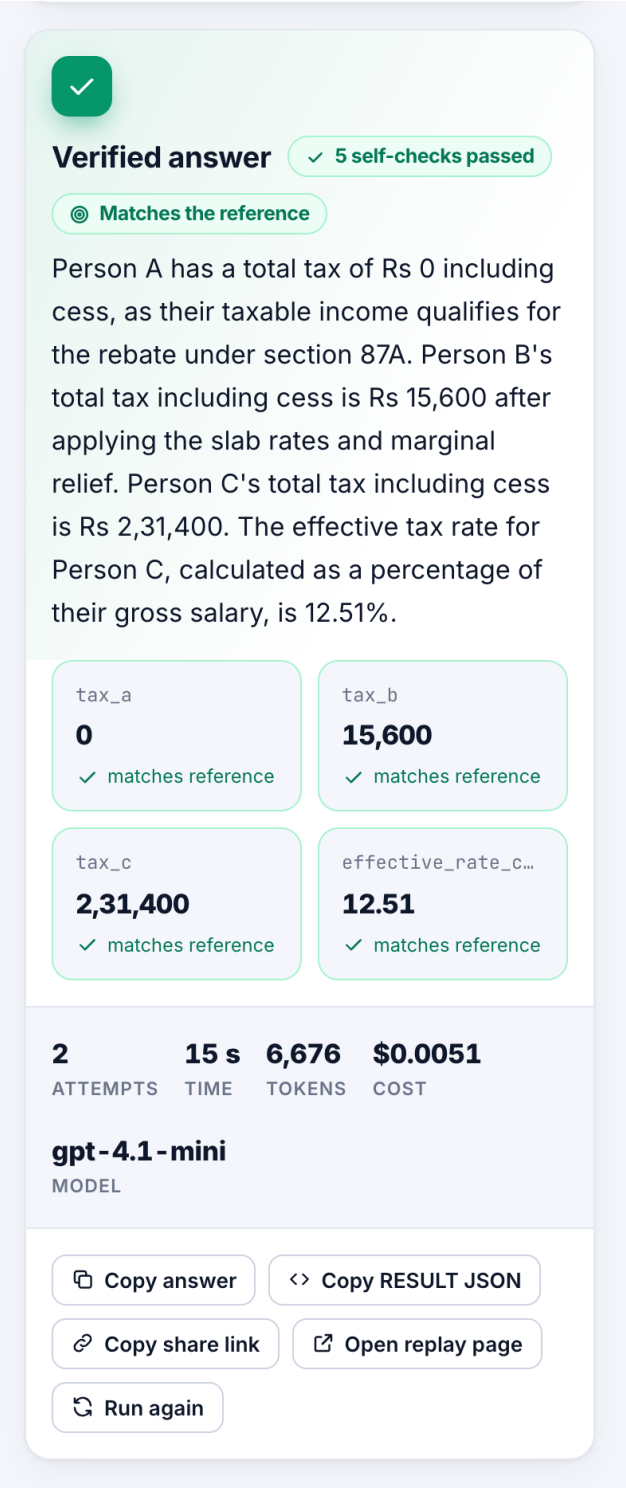


Figure 10. The same final answer on a phone-sized screen (390 px wide).